Pr. BE ELBAGHAZAOUI

ENSA BM

Rapport de TP

TP 2 : Monolithe Modulaire

Restructuration et Préparation à la Migration

Application Monolithique Modulaire

Nom et prénom : BOUHAMIDI-ALAOUI Khaoula
Filière : IACS1
Encadrant : Pr. BE ELBAGHAZAOUI
Établissement : ENSA BM

Table des matières

1 Introduction **2**

2 Nouvelle structure du projet **2**

3 Technologies utilisées **3**

4 Étapes de réalisation **3**
4.1 Ajout de MapStruct ... 3
4.2 Création des DTOs ... 4
4.3 Création du Mapper ... 5
4.4 Refactorisation du Service ... 6
4.5 Adaptation du Controller ... 7

5 Exercice **8**
5.1 Diagramme des modules ... 8
5.2 Module Customer ... 8
5.2.1 Création de l'entité Customer ... 8
5.2.2 Création des DTOs du module Customer ... 9
5.2.3 Création du Repository du module Customer ... 10
5.2.4 Création du Mapper du module Customer ... 10
5.2.5 Création du Service du module Customer ... 11
5.2.6 Création du Controller du module Customer ... 12
5.3 Module Order ... 13
5.3.1 Création de l'entité Order ... 13
5.3.2 Création des DTOs du module Order ... 14
5.3.3 Création du Repository du module Order ... 15
5.3.4 Création du Mapper du module Order ... 15
5.3.5 Création du Service du module Order ... 16
5.3.6 Création du Controller du module Order ... 17
5.3.7 Création d'un client ... 17
5.3.8 Vérification des clients enregistrés ... 18
5.3.9 Création d'un produit ... 19
5.3.10 Vérification des produits enregistrés ... 19
5.3.11 Création d'une commande ... 20
5.3.12 Historique des commandes d'un client ... 21

6 Lancement de l'application **21**

7 Conclusion **22**

1 Introduction

Ce TP consiste à restructurer une application monolithique en modules indépendants (Product, Customer, Order) afin d'améliorer l'organisation du code et préparer une migration vers les microservices.

Commentaire : Cette approche permet de réduire le couplage entre les différentes parties du système.

2 Nouvelle structure du projet

Dans ce TP, nous avons restructuré l'application en plusieurs modules métier distincts afin de mieux organiser le code et définir des responsabilités claires.

La structure adoptée est la suivante :

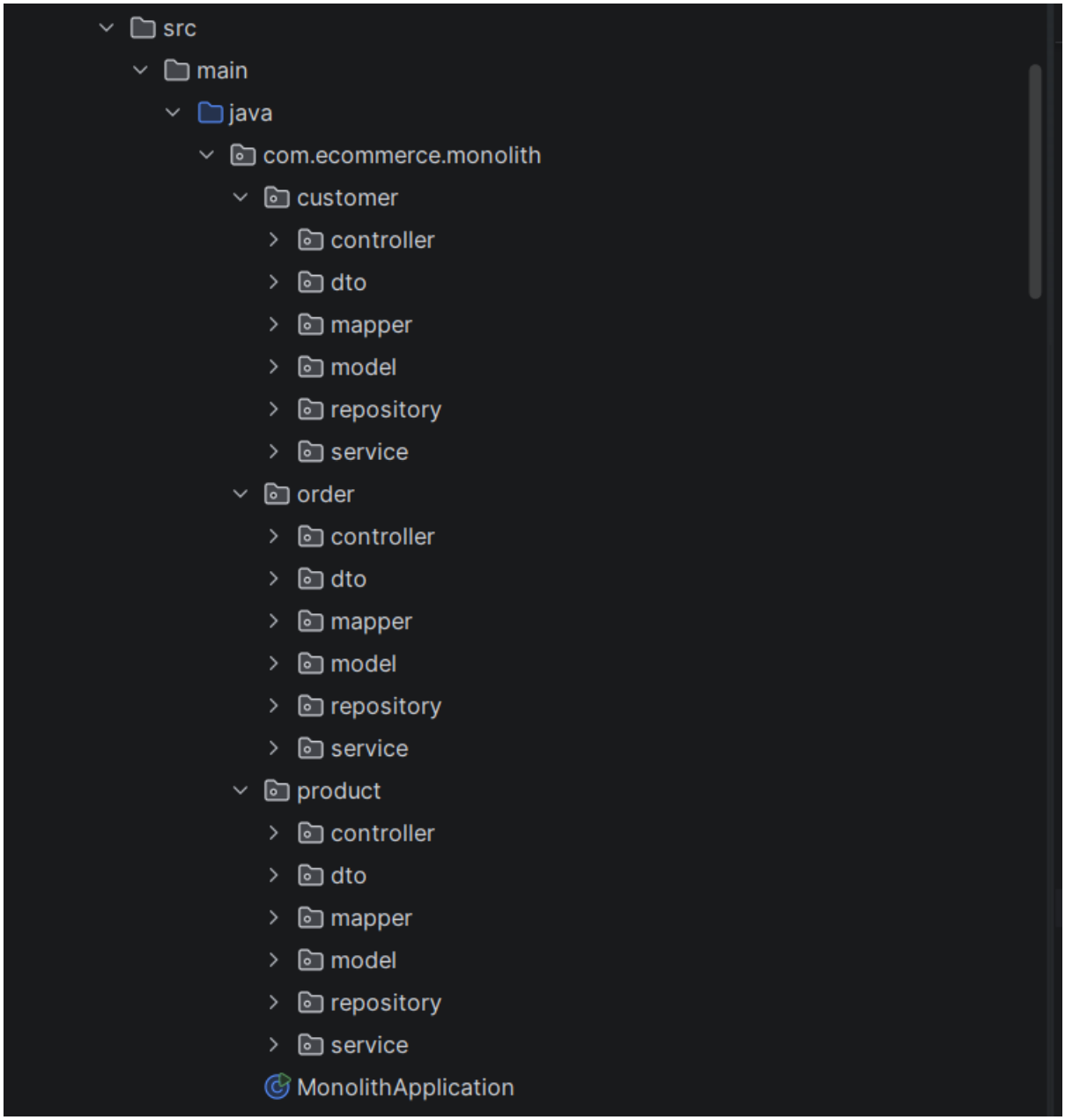


FIGURE 1 – Structure du projet dans IntelliJ

Commentaire : Cette organisation permet de séparer les responsabilités de chaque module et de réduire le couplage entre les différentes parties de l'application.

3 Technologies utilisées

- Spring Boot
- PostgreSQL
- Maven
- IntelliJ IDEA
- Postman
- MapStruct

Commentaire : Ces outils ont été utilisés pour le développement, la persistance des données et les tests.

4 Étapes de réalisation

4.1 Ajout de MapStruct

Ajout de la dépendance MapStruct dans le fichier pom.xml.

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.11.0</version>
            <configuration>
                <source>17</source>
                <target>17</target>
                <annotationProcessorPaths>
                    <path>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </path>
                    <path>
                        <groupId>org.mapstruct</groupId>
                        <artifactId>mapstruct-processor</artifactId>
                        <version>1.6.3</version>
                    </path>
                </annotationProcessorPaths>
            </configuration>
        </plugin>
```

```
<dependencies>
    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct</artifactId>
        <version>1.6.3</version>
    </dependency>
```

FIGURE 2 – Ajout de MapStruct

Commentaire : MapStruct facilite la conversion entre DTO et Entity.

4.2 Création des DTOs

Création des classes ProductDTO et CreateProductRequest.

ProductDTO.java × ProductMapper.java ProductService.j

```
package com.ecommerce.monolith.product.dto;

import lombok.*;
import java.math.BigDecimal;

@Getter  18 usages
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ProductDTO {

    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
}
```

ProductDTO.java ProductMapper.java ProductService.java ProductRepository.java ProductController.java CreateProductRe

```
package com.ecommerce.monolith.product.dto;

import lombok.*;
import java.math.BigDecimal;

@Getter  12 usages
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class CreateProductRequest {

    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
}
```

FIGURE 3 – DTO du module Product

Commentaire : Les DTO permettent de séparer la couche API du modèle de données.

4.3 Création du Mapper

Implémentation du mapper pour la conversion entre objets.

```java
package com.ecommerce.monolith.product.mapper;

import com.ecommerce.monolith.product.dto.CreateProductRequest;
import com.ecommerce.monolith.product.dto.ProductDTO;
import com.ecommerce.monolith.product.model.Product;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.stream.Collectors;

@Component
public class ProductMapper {

    public ProductDTO toDTO(Product product) {  4 usages
        if (product == null) return null;

        ProductDTO dto = new ProductDTO();
        dto.setId(product.getId());
        dto.setName(product.getName());
        dto.setDescription(product.getDescription());
        dto.setPrice(product.getPrice());
        dto.setStock(product.getStock());
        return dto;
    }

    public List<ProductDTO> toDTOList(List<Product> products) {  1 usage
        return products.stream()  Stream<Product>
                .map(this::toDTO)  Stream<ProductDTO>
                .collect(Collectors.toList());
    }
```

```java
    public Product toEntity(CreateProductRequest request) {  1 usage
        if (request == null) return null;

        Product product = new Product();
        product.setName(request.getName());
        product.setDescription(request.getDescription());
        product.setPrice(request.getPrice());
        product.setStock(request.getStock());
        return product;
    }

    public void updateEntity(CreateProductRequest request, Product product) {  1 usage
        if (request == null || product == null) return;

        product.setName(request.getName());
        product.setDescription(request.getDescription());
        product.setPrice(request.getPrice());
        product.setStock(request.getStock());
    }
}
```

FIGURE 4 – Mapper du module Product

Commentaire : Le mapper centralise la transformation des données.

4.4 Refactorisation du Service

Séparation en interface et implémentation.

ProductDTO.java ProductMapper.java ProductService.java ProductRepository.java ProductCo

```java
package com.ecommerce.monolith.product.service;

import com.ecommerce.monolith.product.dto.CreateProductRequest;
import com.ecommerce.monolith.product.dto.ProductDTO;

import java.util.List;

public interface ProductService {  3 usages  1 implementation

    List<ProductDTO> getAllProducts();  1 usage  1 implementation

    ProductDTO getProductById(Long id);  1 usage  1 implementation

    ProductDTO createProduct(CreateProductRequest request);  1 usage  1 implementation

    ProductDTO updateProduct(Long id, CreateProductRequest request);  1 usage  1 implementation

    void deleteProduct(Long id);  1 usage  1 implementation
}
```

ProductServiceImpl.java ProductDTO.java ProductMapper.java Product

```java
package com.ecommerce.monolith.product.service;

import com.ecommerce.monolith.product.dto.CreateProductRequest;
import com.ecommerce.monolith.product.dto.ProductDTO;
import com.ecommerce.monolith.product.mapper.ProductMapper;
import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@RequiredArgsConstructor
@Transactional
public class ProductServiceImpl implements ProductService {

    private final ProductRepository repository;
    private final ProductMapper mapper;

    @Override  1 usage
    @Transactional(readOnly = true)
    public List<ProductDTO> getAllProducts() {
        return mapper.toDTOList(repository.findAll());
    }

    @Override  1 usage
    @Transactional(readOnly = true)
    public ProductDTO getProductById(Long id) {
```

FIGURE 5 – Service du module Product

Commentaire : Cette structure améliore la maintenabilité.

4.5 Adaptation du Controller

Utilisation des DTO au lieu des entités.

ProductController.java × ProductServiceImpl.java ProductDTO.java ProductMapper.java ProductService.ja

```java
package com.ecommerce.monolith.product.controller;

import com.ecommerce.monolith.product.dto.CreateProductRequest;
import com.ecommerce.monolith.product.dto.ProductDTO;
import com.ecommerce.monolith.product.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    @GetMapping
    public ResponseEntity<List<ProductDTO>> getAll() {
        return ResponseEntity.ok(productService.getAllProducts());
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }

    @PostMapping
    public ResponseEntity<ProductDTO> create(@RequestBody CreateProductRequest request) {
```

```java
    @PostMapping
    public ResponseEntity<ProductDTO> create(@RequestBody CreateProductRequest request) {
        return ResponseEntity.status(201).body(productService.createProduct(request));
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductDTO> update(@PathVariable Long id,
                                             @RequestBody CreateProductRequest request) {
        return ResponseEntity.ok(productService.updateProduct(id, request));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

FIGURE 6 – Controller du module Product

Commentaire : L'API devient plus propre et sécurisée

5 Exercice

5.1 Diagramme des modules

[Product Module] --->

\

> [Order Module]

/

[Customer Module] --->

FIGURE 7 – Diagramme des dépendances entre modules

Commentaire : Le module Order dépend de Product et Customer à travers les services publics.

5.2 Module Customer

5.2.1 Création de l'entité Customer

Nous avons créé l'entité Customer pour représenter les clients dans la base de données.

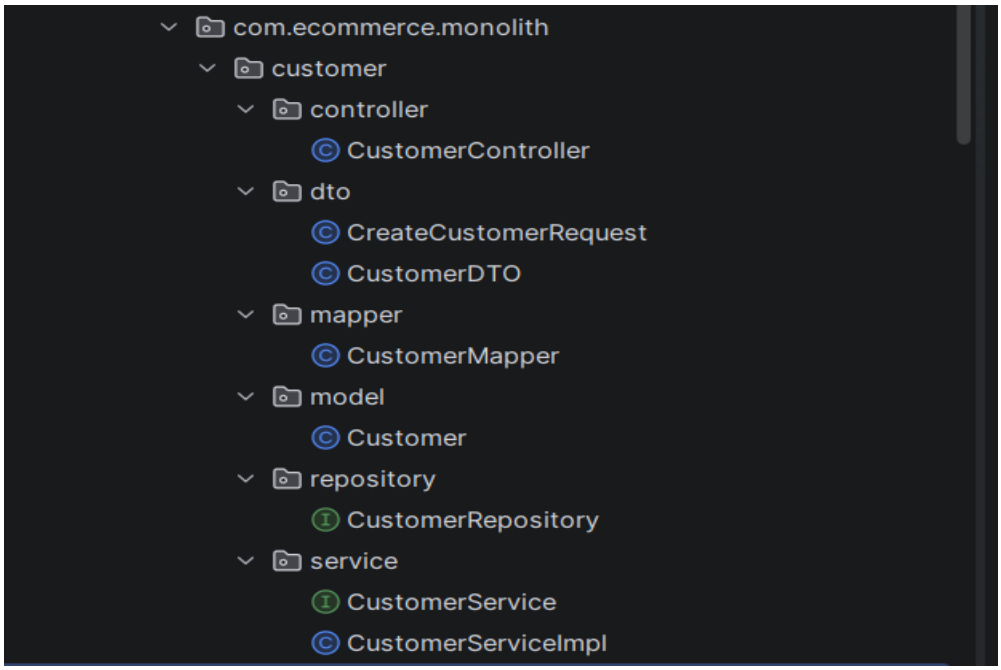


FIGURE 7 – Entité Customer

Commentaire : Cette entité contient les informations principales du client, comme l'identifiant, le nom complet et l'adresse email.

5.2.2 Création des DTOs du module Customer

Nous avons créé les classes CustomerDTO et CreateCustomerRequest.

CustomerDTO.java | CreateCustomerRequest.java

```java
package com.ecommerce.monolith.customer.dto;

import lombok.*;

@Getter  no usages
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class CustomerDTO {
    private Long id;
    private String fullName;
    private String email;
}
```

CreateCustomerRequest.java | CustomerRepository.ja

```java
package com.ecommerce.monolith.customer.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter  no usages
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class CreateCustomerRequest {
    private String fullName;
    private String email;
}
```

FIGURE 8 – DTOs du module Customer

Commentaire : Le DTO permet d'exposer les données du client, tandis que la classe Request permet de recevoir les données envoyées par le client.

5.2.3 Création du Repository du module Customer

Nous avons créé CustomerRepository pour permettre l'accès aux données du module Customer.

```java
package com.ecommerce.monolith.customer.repository;

import com.ecommerce.monolith.customer.model.Customer;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CustomerRepository extends JpaRepository<Customer, Long> {
}
```

FIGURE 9 – Repository du module Customer

Commentaire : Le repository hérite de JpaRepository afin de simplifier les opérations CRUD.

5.2.4 Création du Mapper du module Customer

Nous avons créé CustomerMapper pour convertir entre entité et DTO.

```java
package com.ecommerce.monolith.customer.mapper;

import com.ecommerce.monolith.customer.dto.CreateCustomerRequest;
import com.ecommerce.monolith.customer.dto.CustomerDTO;
import com.ecommerce.monolith.customer.model.Customer;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.stream.Collectors;

@Component
public class CustomerMapper {

    public CustomerDTO toDTO(Customer customer) {
        if (customer == null) return null;

        CustomerDTO dto = new CustomerDTO();
        dto.setId(customer.getId());
        dto.setFullName(customer.getFullName());
        dto.setEmail(customer.getEmail());
        return dto;
    }

    public List<CustomerDTO> toDTOList(List<Customer> customers) {
        return customers.stream()
                .map(this::toDTO)
                .collect(Collectors.toList());
    }

    public Customer toEntity(CreateCustomerRequest request) {
```

```java
                .collect(Collectors.toList());
    }

    public Customer toEntity(CreateCustomerRequest request) {  1 usage
        if (request == null) return null;

        Customer customer = new Customer();
        customer.setFullName(request.getFullName());
        customer.setEmail(request.getEmail());
        return customer;
    }

    public void updateEntity(CreateCustomerRequest request, Customer customer) {  no usages
        if (request == null || customer == null) return;

        customer.setFullName(request.getFullName());
        customer.setEmail(request.getEmail());
    }
}
```

FIGURE 10 – Mapper du module Customer

Commentaire : Le mapper du module Customer centralise les transformations entre les objets du domaine et les objets de transport.

5.2.5 Création du Service du module Customer

Nous avons implémenté CustomerService et CustomerServiceImpl.

```java
CustomerMapper.java   CustomerService.java   CustomerServiceImpl.java

package com.ecommerce.monolith.customer.service;

import com.ecommerce.monolith.customer.dto.CreateCustomerRequest;
import com.ecommerce.monolith.customer.dto.CustomerDTO;

import java.util.List;

public interface CustomerService {  no usages  1 implementation

    List<CustomerDTO> getAllCustomers();  no usages  1 implementation

    CustomerDTO getCustomerById(Long id);  no usages  1 implementation

    CustomerDTO createCustomer(CreateCustomerRequest request);  no usages  1

    boolean existsById(Long id);  no usages  1 implementation
}
```

CustomerMapper.java | CustomerService.java | CustomerServiceImpl.java | Order.java | OrderDTO.java

```java
package com.ecommerce.monolith.customer.service;

import com.ecommerce.monolith.customer.dto.CreateCustomerRequest;
import com.ecommerce.monolith.customer.dto.CustomerDTO;
import com.ecommerce.monolith.customer.mapper.CustomerMapper;
import com.ecommerce.monolith.customer.model.Customer;
import com.ecommerce.monolith.customer.repository.CustomerRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class CustomerServiceImpl implements CustomerService {

    private final CustomerRepository repository;
    private final CustomerMapper mapper;

    @Override  no usages
    public List<CustomerDTO> getAllCustomers() {
        return mapper.toDTOList(repository.findAll());
    }

    @Override  no usages
    public CustomerDTO getCustomerById(Long id) {
        Customer customer = repository.findById(id)
                .orElseThrow(() -> new RuntimeException("Customer not found with id: " + id));
        return mapper.toDTO(customer);
    }

    @Override  no usages
    public CustomerDTO createCustomer(CreateCustomerRequest request) {
        Customer customer = mapper.toEntity(request);
        Customer saved = repository.save(customer);
        return mapper.toDTO(saved);
    }

    @Override  no usages
    public boolean existsById(Long id) {
        return repository.existsById(id);
    }
}
```

FIGURE 11 – Service du module Customer

Commentaire : Le service du module Customer contient les méthodes permettant de récupérer, créer et vérifier l'existence d'un client.

5.2.6 Création du Controller du module Customer

Nous avons créé CustomerController pour exposer les endpoints REST du module Customer.

```java
package com.ecommerce.monolith.customer.controller;

import com.ecommerce.monolith.customer.dto.CreateCustomerRequest;
import com.ecommerce.monolith.customer.dto.CustomerDTO;
import com.ecommerce.monolith.customer.service.CustomerService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/customers")
@RequiredArgsConstructor
public class CustomerController {

    private final CustomerService customerService;

    @GetMapping
    public List<CustomerDTO> getAll() {
        return customerService.getAllCustomers();
    }

    @GetMapping("/{id}")
    public CustomerDTO getById(@PathVariable Long id) {
        return customerService.getCustomerById(id);
    }

    @PostMapping
    public CustomerDTO create(@RequestBody CreateCustomerRequest request) {
        return customerService.createCustomer(request);
```

FIGURE 12 – Controller du module Customer

Commentaire : Ce contrôleur permet de créer et consulter les clients via des requêtes HTTP.

5.3 Module Order

5.3.1 Création de l'entité Order

Nous avons créé l'entité Order pour représenter les commandes.

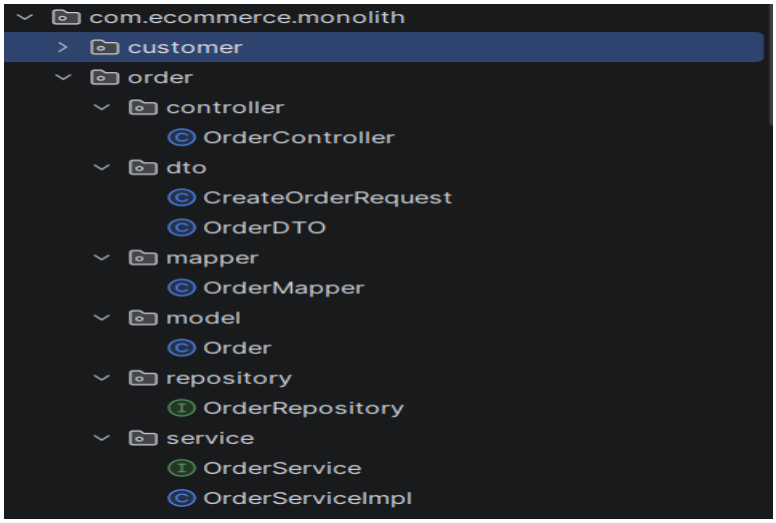


FIGURE 13 – Entité Order

Commentaire : Une commande contient l'identifiant du client, l'identifiant du produit ainsi que la quantité commandée.

5.3.2 Création des DTOs du module Order

Nous avons créé OrderDTO et CreateOrderRequest.

OrderDTO.java | CreateOrderRequest.java | OrderRepository.java

```java
package com.ecommerce.monolith.order.dto;

import lombok.*;

@Getter  no usages
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class OrderDTO {
    private Long id;
    private Long customerId;
    private Long productId;
    private Integer quantity;
}
```

OrderDTO.java | CreateOrderRequest.java | OrderRepository.java

```java
package com.ecommerce.monolith.order.dto;

import lombok.*;

@Getter  no usages
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class CreateOrderRequest {
    private Long customerId;
    private Long productId;
    private Integer quantity;
}
```

FIGURE 14 – DTOs du module Order

Commentaire : Ces classes servent à transporter les données de commande sans exposer directement l'entité Order.

5.3.3 Création du Repository du module Order

Nous avons créé OrderRepository pour gérer la persistance des commandes.\

```java
package com.ecommerce.monolith.order.repository;

import com.ecommerce.monolith.order.model.Order;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByCustomerId(Long customerId);
}
```

FIGURE 15 – Repository du module Order

Commentaire : Le repository du module Order contient également une méthode permettant de retrouver les commandes d'un client donné.

5.3.4 Création du Mapper du module Order

Nous avons créé OrderMapper pour convertir les objets du module Order.

```java
package com.ecommerce.monolith.order.mapper;

import com.ecommerce.monolith.order.dto.CreateOrderRequest;
import com.ecommerce.monolith.order.dto.OrderDTO;
import com.ecommerce.monolith.order.model.Order;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.stream.Collectors;

@Component
public class OrderMapper {

    public OrderDTO toDTO(Order order) {
        if (order == null) return null;

        OrderDTO dto = new OrderDTO();
        dto.setId(order.getId());
        dto.setCustomerId(order.getCustomerId());
        dto.setProductId(order.getProductId());
        dto.setQuantity(order.getQuantity());
        return dto;
    }

    public List<OrderDTO> toDTOList(List<Order> orders) {
        return orders.stream()
                .map(this::toDTO)
                .collect(Collectors.toList());
    }
```

```java
    public Order toEntity(CreateOrderRequest request) {  1 usage
        if (request == null) return null;

        Order order = new Order();
        order.setCustomerId(request.getCustomerId());
        order.setProductId(request.getProductId());
        order.setQuantity(request.getQuantity());
        return order;
    }
}
```

FIGURE 16 – Mapper du module Order

Commentaire : Ce mapper permet de transformer les données de commande entre la couche métier et la couche API.

5.3.5 Création du Service du module Order

Nous avons implémenté OrderService et OrderServiceImpl.

OrderService.java | OrderMapper.java | OrderDTO.java | CreateOrderRequest.java | OrderRepository.java

```java
package com.ecommerce.monolith.order.service;

import com.ecommerce.monolith.order.dto.CreateOrderRequest;
import com.ecommerce.monolith.order.dto.OrderDTO;

import java.util.List;

public interface OrderService {  3 usages  1 implementation

    OrderDTO createOrder(CreateOrderRequest request);  1 usage  1 implementation

    List<OrderDTO> getCustomerOrderHistory(Long customerId);  1 usage  1 implementation
}
```

OrderServiceImpl.java | OrderMapper.java | OrderDTO.java | CreateOrderRequest.java | OrderRepository.ja

```java
package com.ecommerce.monolith.order.service;

import com.ecommerce.monolith.customer.service.CustomerService;
import com.ecommerce.monolith.order.dto.CreateOrderRequest;
import com.ecommerce.monolith.order.dto.OrderDTO;
import com.ecommerce.monolith.order.mapper.OrderMapper;
import com.ecommerce.monolith.order.model.Order;
import com.ecommerce.monolith.order.repository.OrderRepository;
import com.ecommerce.monolith.product.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class OrderServiceImpl implements OrderService {

    private final OrderRepository repository;
    private final OrderMapper mapper;
    private final CustomerService customerService;
    private final ProductService productService;

    @Override  1 usage
    public OrderDTO createOrder(CreateOrderRequest request) {
        if (!customerService.existsById(request.getCustomerId())) {
            throw new RuntimeException("Customer not found");
        }
```

FIGURE 17 – Service du module Order

Commentaire : Le service du module Order utilise les services publics de Product et Customer afin de vérifier l'existence des entités avant la création d'une commande.

5.3.6 Création du Controller du module Order

Nous avons créé OrderController pour exposer les endpoints du module Order.

OrderMapper.java | OrderDTO.java | CreateOrderRequest.java | OrderRepository.java | OrderController.java

```java
package com.ecommerce.monolith.order.controller;

import com.ecommerce.monolith.order.dto.CreateOrderRequest;
import com.ecommerce.monolith.order.dto.OrderDTO;
import com.ecommerce.monolith.order.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @PostMapping
    public OrderDTO create(@RequestBody CreateOrderRequest request) {
        return orderService.createOrder(request);
    }

    @GetMapping("/customer/{customerId}")
    public List<OrderDTO> getCustomerHistory(@PathVariable Long customerId) {
        return orderService.getCustomerOrderHistory(customerId);
    }
}
```

FIGURE 18 – Controller du module Order

Commentaire : Le contrôleur permet de créer une commande et de consulter l'historique des commandes d'un client.

5.3.7 Création d'un client

Avant de créer une commande, nous avons ajouté un client via l'API du module Customer.

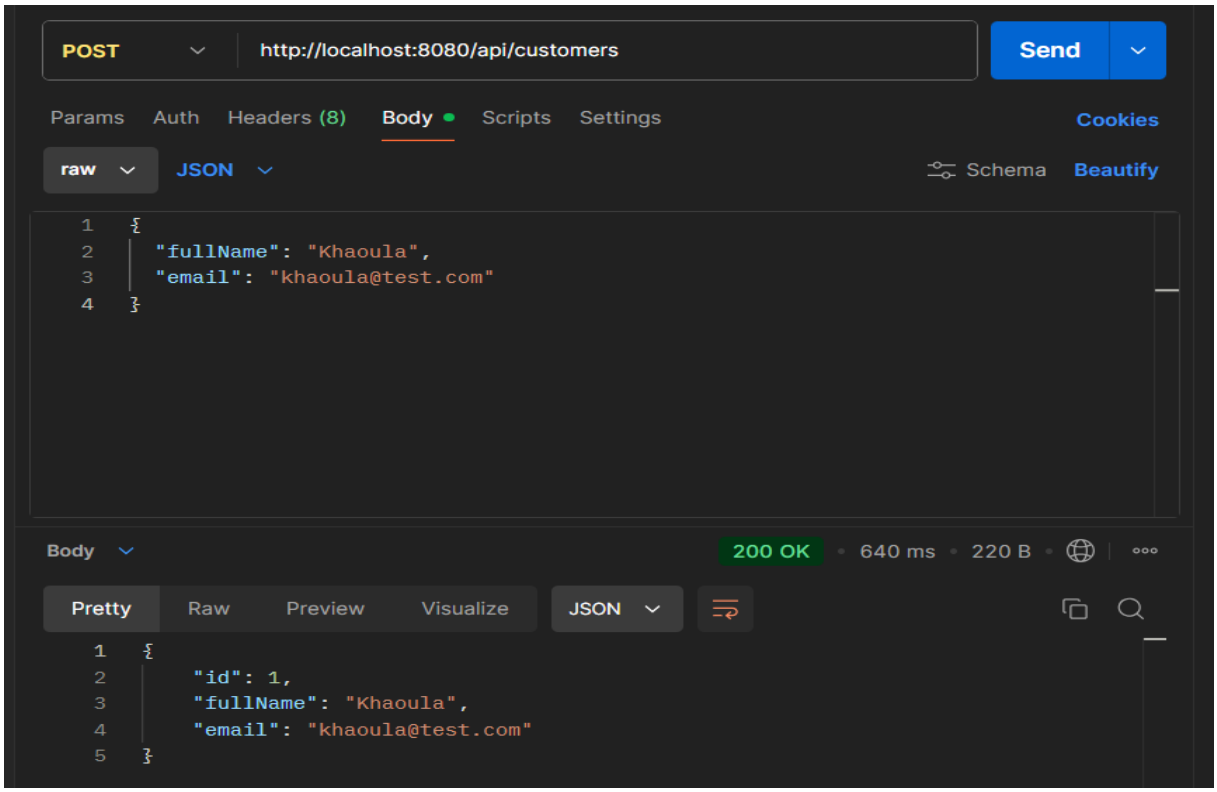


FIGURE 19 – POST /api/customers

Commentaire : Cette étape permet d'obtenir un identifiant client valide qui sera utilisé dans la commande.

5.3.8 Vérification des clients enregistrés

Nous avons ensuite vérifié la liste des clients enregistrés.

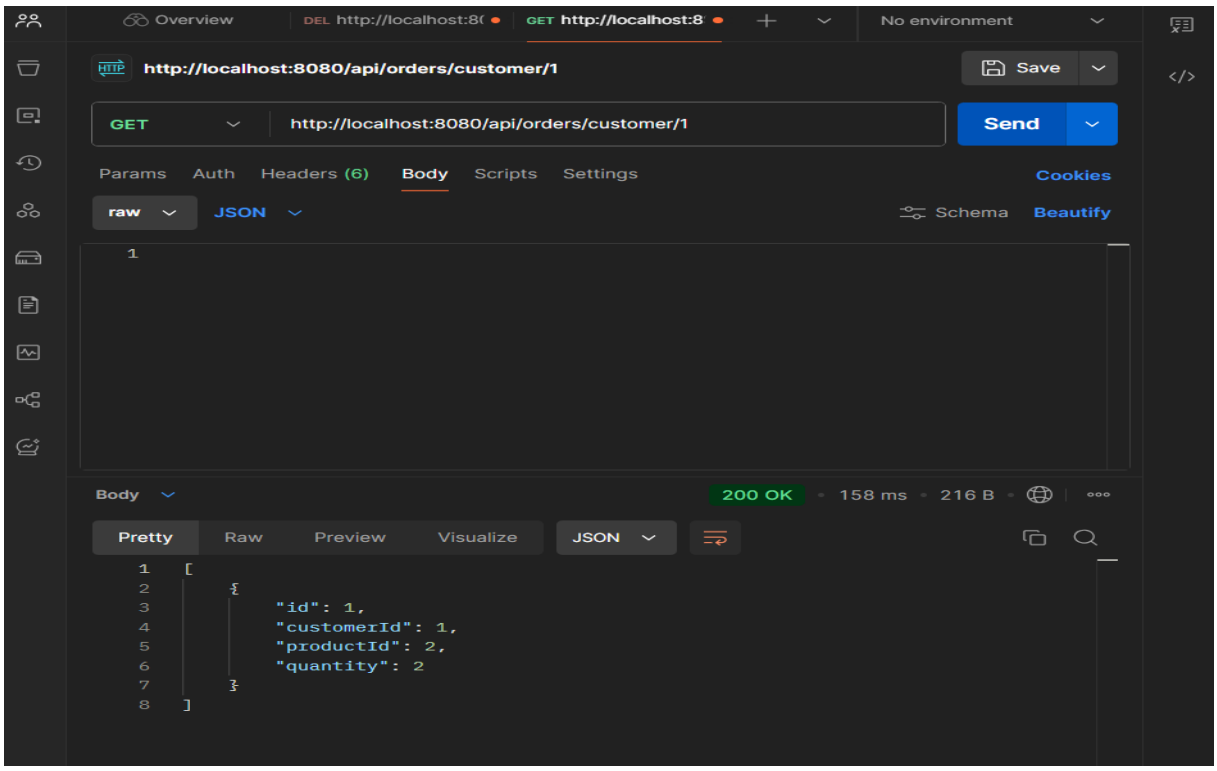


FIGURE 20 – GET /api/customers

Commentaire : Cette vérification confirme que le client a bien été sauvegardé dans la base de données.

5.3.9 Création d'un produit

Nous avons également ajouté un produit via l'API du module Product.

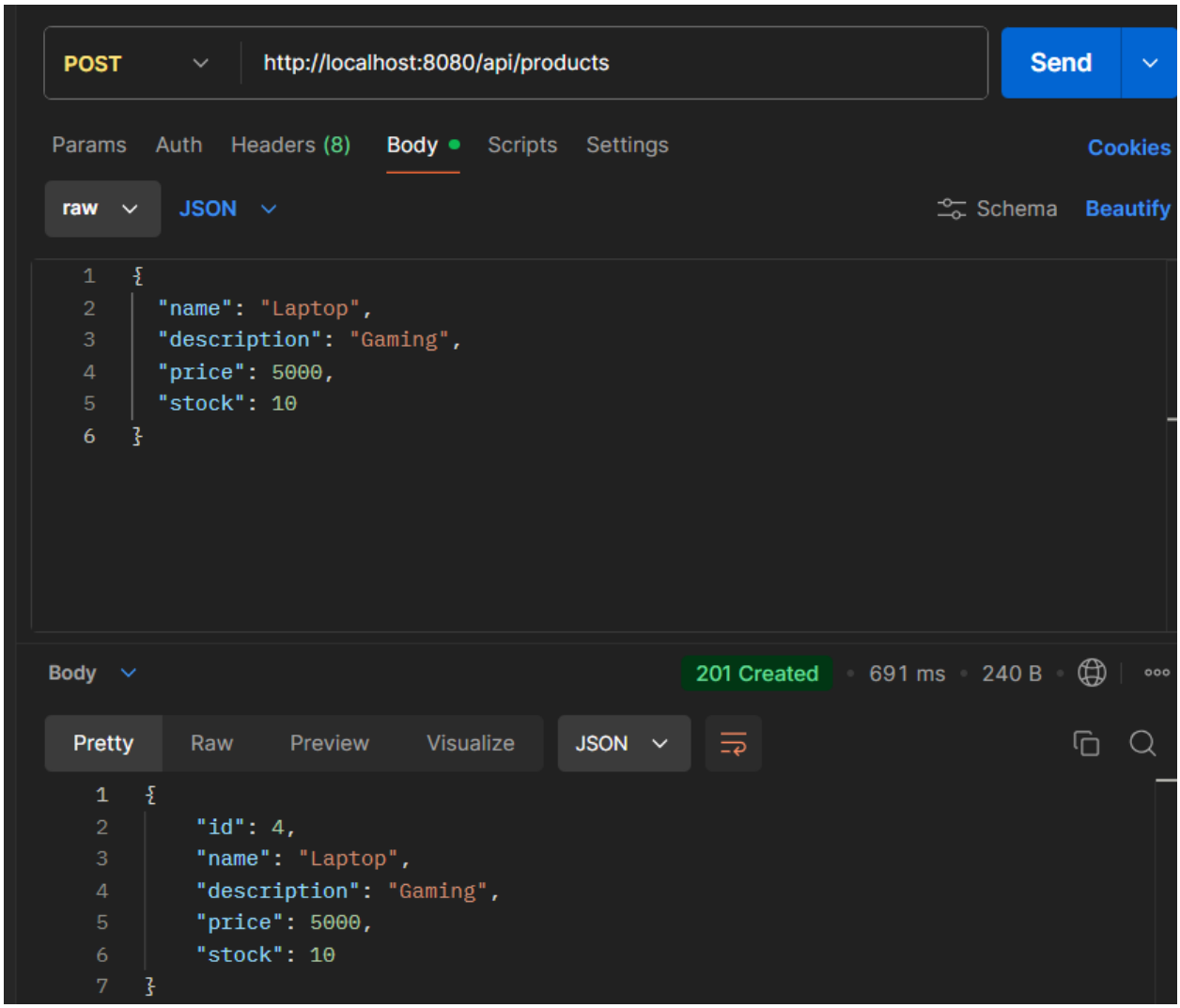


FIGURE 21 – POST /api/products

Commentaire : Cette étape est nécessaire pour disposer d'un identifiant produit valide avant la création d'une commande.

5.3.10 Vérification des produits enregistrés

Nous avons ensuite vérifié les produits disponibles dans la base.

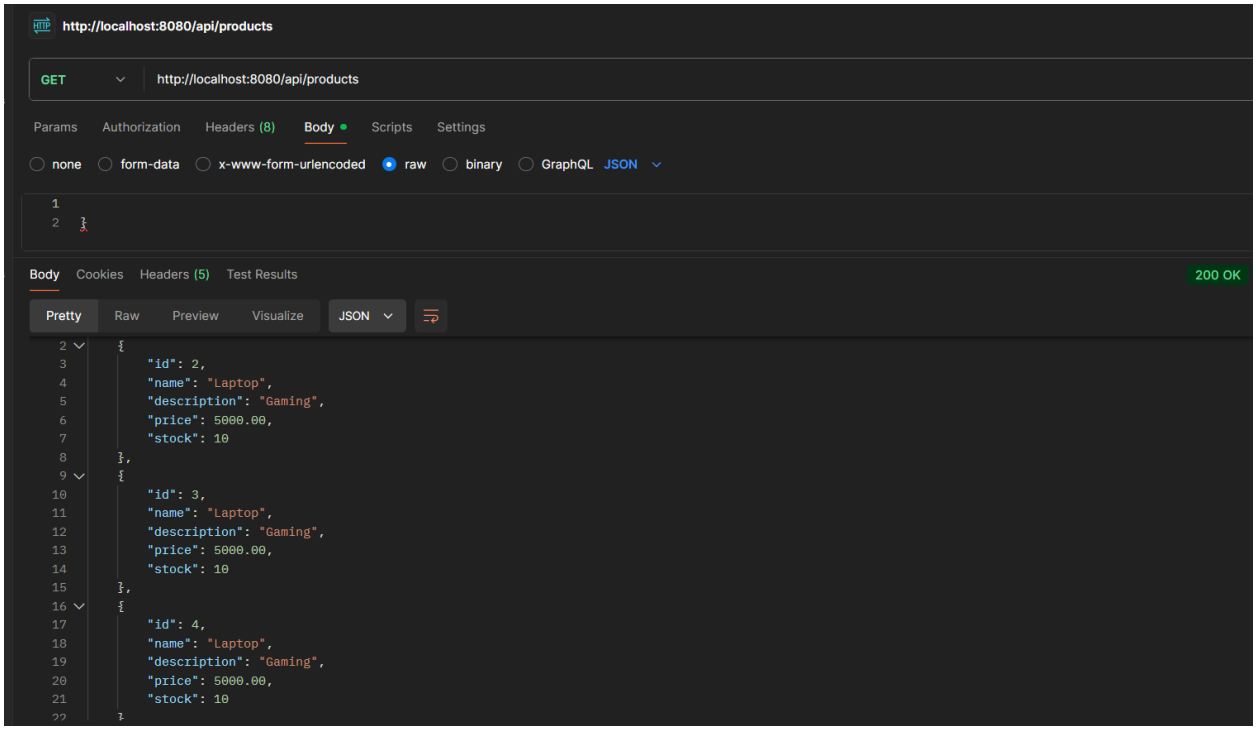


FIGURE 22 – GET /api/products

Commentaire : Cette vérification a montré que l'identifiant produit utilisé dans la commande devait correspondre à un produit existant.

5.3.11 Création d'une commande

Nous avons créé une commande via Postman en utilisant les identifiants valides du client et du produit.

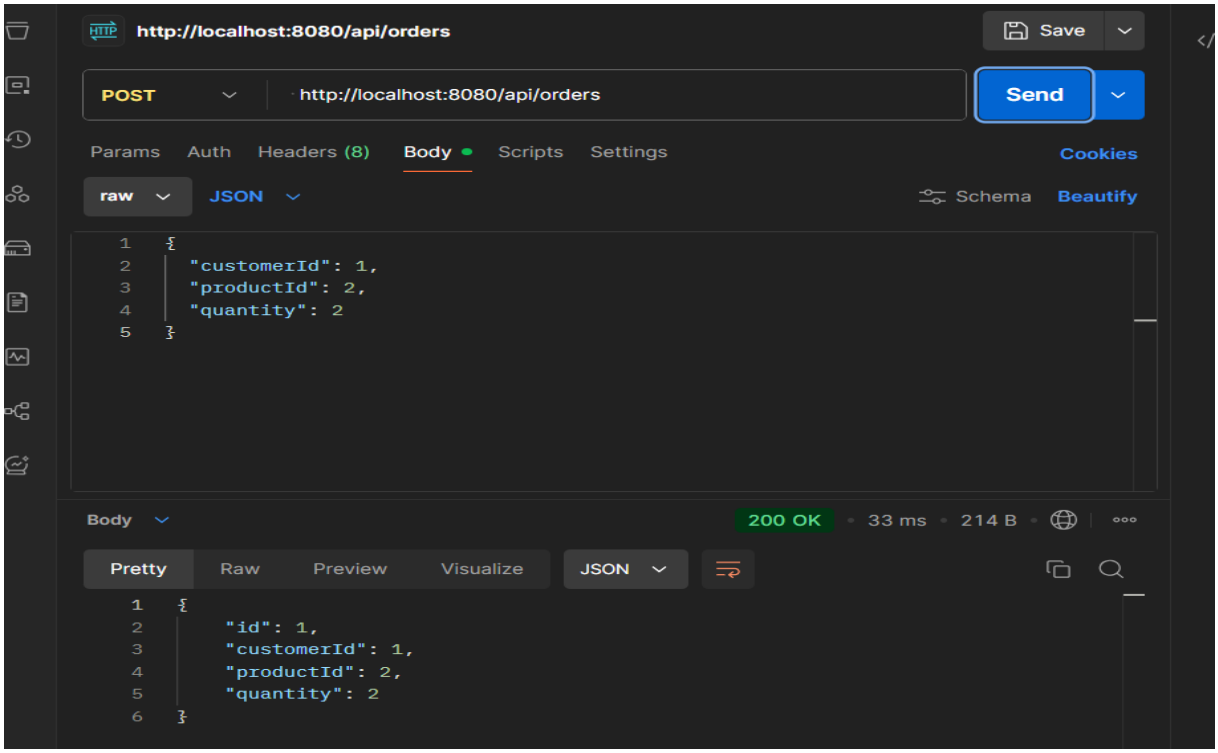


FIGURE 23 – POST /api/orders

Commentaire : Une erreur 500 a d'abord été rencontrée à cause d'un identifiant produit invalide, puis l'erreur a été corrigée en utilisant un identifiant produit existant.

5.3.12 Historique des commandes d'un client

Nous avons testé l'endpoint GET /api/orders/customer/{id} pour récupérer les commandes liées à un client donné.

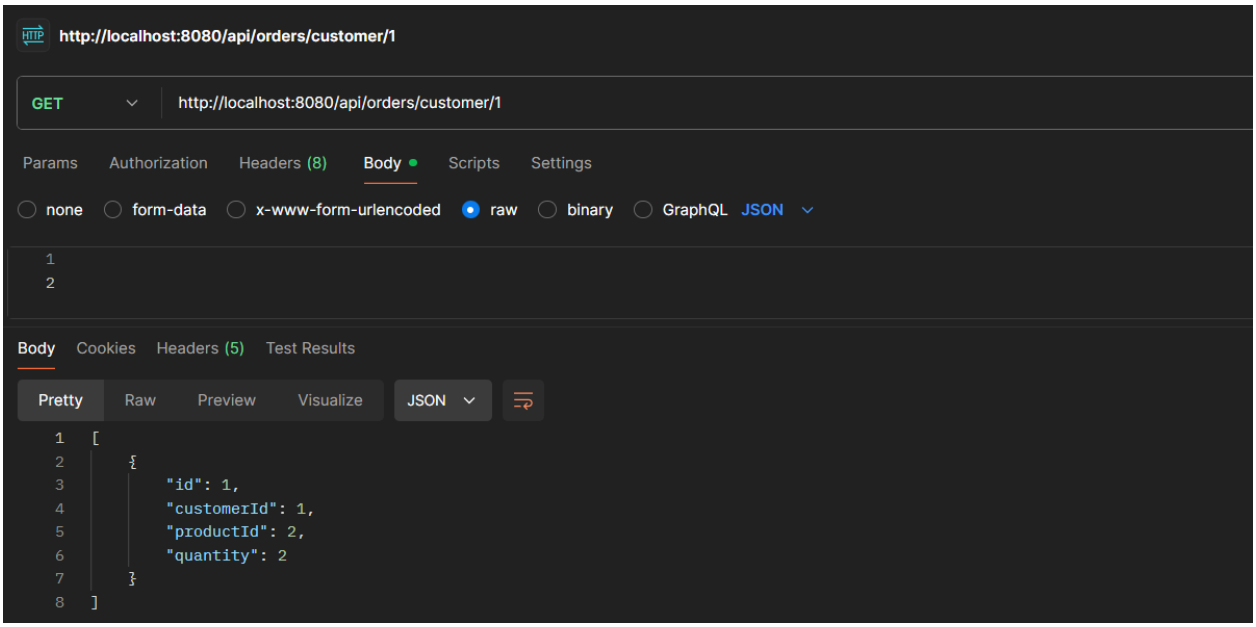
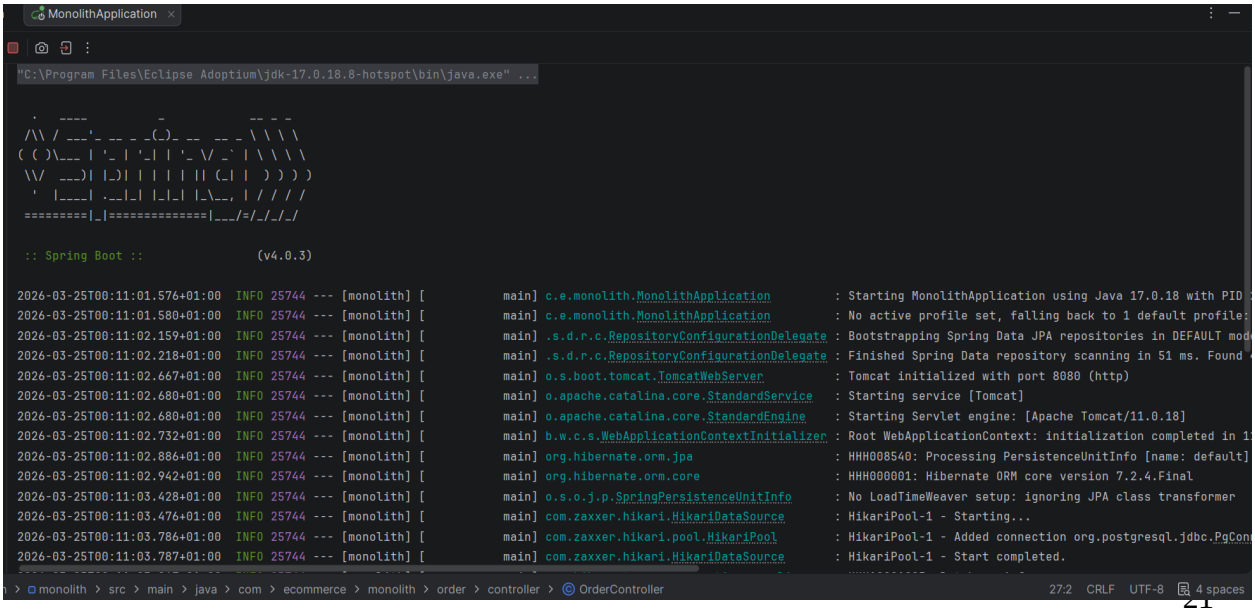


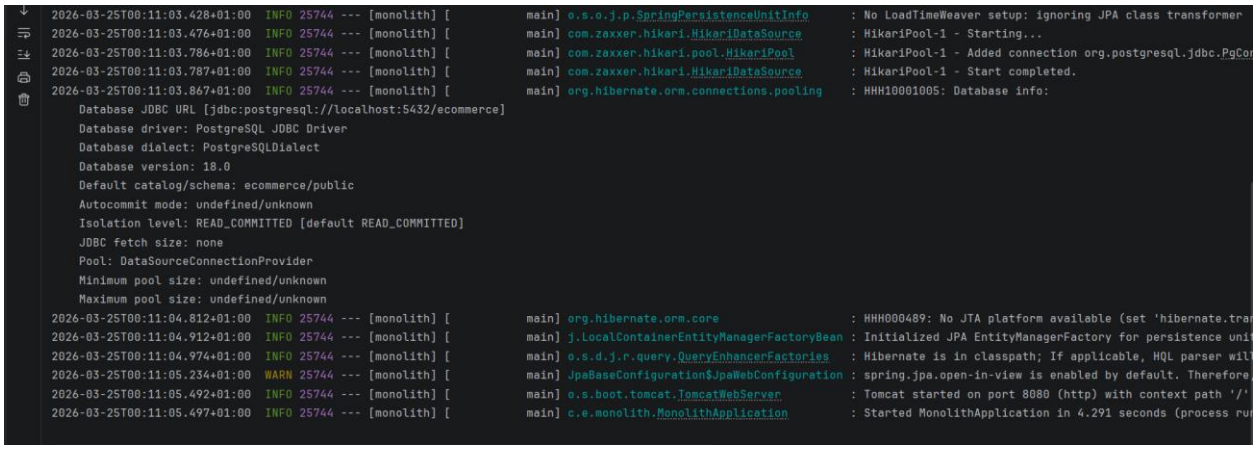
FIGURE 24 – GET /api/orders/customer/{id}

Commentaire : Le résultat obtenu confirme que le module Order récupère correctement l'historique des commandes du client demandé.

6 Lancement de l'application

Après l'implémentation complète des modules Product, Customer et Order, l'application a été lancée avec succès.





```
2026-03-25T00:11:03.428+01:00  INFO 25744 --- [monolith] [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-03-25T00:11:03.476+01:00  INFO 25744 --- [monolith] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-03-25T00:11:03.786+01:00  INFO 25744 --- [monolith] [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection org.postgresql.jdbc.PgCon
2026-03-25T00:11:03.787+01:00  INFO 25744 --- [monolith] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-03-25T00:11:03.867+01:00  INFO 25744 --- [monolith] [           main] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
    Database JDBC URL [jdbc:postgresql://localhost:5432/ecommerce]
    Database driver: PostgreSQL JDBC Driver
    Database dialect: PostgreSQLDialect
    Database version: 18.0
    Default catalog/schema: ecommerce/public
    Autocommit mode: undefined/unknown
    Isolation level: READ_COMMITTED [default READ_COMMITTED]
    JDBC fetch size: none
    Pool: DataSourceConnectionProvider
    Minimum pool size: undefined/unknown
    Maximum pool size: undefined/unknown
2026-03-25T00:11:04.812+01:00  INFO 25744 --- [monolith] [           main] org.hibernate.orm.core                   : HHH000489: No JTA platform available (set 'hibernate.tran
2026-03-25T00:11:04.912+01:00  INFO 25744 --- [monolith] [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit
2026-03-25T00:11:04.974+01:00  INFO 25744 --- [monolith] [           main] o.s.d.j.r.query.QueryEnhancerFactories   : Hibernate is in classpath; If applicable, HQL parser will
2026-03-25T00:11:05.234+01:00  WARN 25744 --- [monolith] [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore,
2026-03-25T00:11:05.492+01:00  INFO 25744 --- [monolith] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path '/'
2026-03-25T00:11:05.497+01:00  INFO 25744 --- [monolith] [           main] c.e.monolith.MonolithApplication         : Started MonolithApplication in 4.291 seconds (process run
```

FIGURE 25 – Console de démarrage de l'application

Commentaire : Le démarrage réussi du serveur confirme le bon fonctionnement global de l'application après restructuration.

7 Conclusion

Ce TP nous a permis de restructurer une application en architecture modulaire, d'utiliser les DTOs et les mappers, et d'implémenter plusieurs modules métier interconnectés.

Commentaire : Cette architecture améliore l'organisation du projet et constitue une